

■ 강의명: CSCI103

■ 주차: Lecture 10

■ 교수명: UIUC Student

■ 목적: 본 문서는 CSCI103 Lecture 10 강의 자료를 바탕으로 작성되었습니다. 강의는 데이터와 모델에 대한 심층적인 고려사항을 다루며, 특히 모델 편향, 데이터 불균형, 익명성 문제를 강조합니다. AutoML의 개념과 Databricks의 글래스박스(Glassbox) AutoML 구현을 통해 모델 개발의 효율성과 투명성을 탐구합니다. 또한, 대규모 신용카드 사기 탐지 시스템의 실제 사례를 통해 복잡한 비즈니스 요구사항과 기술적 해결책을 살펴봅니다. 이 문서는 비전공자도 쉽게 이해할 수 있도록 핵심 개념, 원리, 절차, 그리고 실제 적용 사례를 상세히 설명합니다.

Contents

용어 정리

다음 표는 강의에서 다루는 주요 용어들을 쉽게 설명하고 원어를 병기하여 이해를 돕습니다.

Table 1: 주요 용어 정리

용어	쉬운 설명	원어	비고
AutoML	기계 학습 모델 개발 과정을 자동화하는 기술	Automated Machine Learning	데이터만 입력하면 모델을 자동으로 생성
블랙박스 ML	모델의 내부 작동 방식을 알 수 없는 ML 시스템	Blackbox ML	결과는 알지만, 왜 그런 결과가 나왔는지 설명하기 어려움
글래스박스 ML	모델의 내부 작동 방식이 투명하게 공개되는 ML 시스템	Glassbox ML	모든 과정(하이퍼파라미터 튜닝, EDA 등)을 검토 가능
SHAP	모델의 예측에 각 특징이 얼마나 기여했는지 설명하는 방법	SHapley Additive exPlanations	모델 해석 가능성(Interpretability)을 높임
편향 (Bias)	모델의 예측값과 실제값의 차이. 주로 과소적합(Underfitting)으로 발생	Bias	모델이 데이터의 패턴을 충분히 학습하지 못함
분산 (Variance)	모델이 훈련 데이터의 작은 변화에도 민감하게 반응하는 정도. 과대적합(Overfitting)으로 발생	Variance	모델이 훈련 데이터에 너무 맞춰져 새로운 데이터에 약함
과소적합	모델이 훈련 데이터를 충분히 학습하지 못해 성능이 낮은 상태	Underfitting	편향이 높음
과대적합	모델이 훈련 데이터에 너무 맞춰져 새로운 데이터에 대한 일반화 성능이 낮은 상태	Overfitting	분산이 높음
데이터 불균형	특정 클래스의 데이터 수가 다른 클래스에 비해 현저히 적은 상태	Imbalanced Data	사기 탐지, 질병 진단 등에서 흔히 발생
SMOTE	소수 클래스의 합성 샘플을 생성하여 데이터 불균형을 해소하는 기법	Synthetic Minority Over-sampling Technique	오버샘플링 기법 중 하나
람다 아키텍처	배치 처리와 실시간 스트리밍 처리를 결합한 데이터 처리 아키텍처	Lambda Architecture	대규모 데이터 처리 시스템에 사용
카산드라	높은 확장성과 가용성을 제공하는 NoSQL 데이터베이스	Cassandra	대규모 분산 환경에 적합
PII	개인을 식별할 수 있는 모든 정보	Personally Identifiable Information	주민등록번호, 전화번호, 이메일 주소 등
특징 공학	원시 데이터에서 모델 학습에 유용한 특징을 추출하고 변환하는 과정	Feature Engineering	모델 성능에 큰 영향을 미침
순환 버퍼	고정된 크기의 버퍼를 사용하여 데이터를 효율적으로 저장하고 관리하는 자료 구조	Circular Buffer / Ring Buffer	오래된 데이터를 자동으로 덮어쓰며 최신 데이터 유지

핵심 개념 및 원리

데이터 및 모델 관련 주요 고려사항

기계 학습(ML) 모델의 성공은 단순히 복잡한 알고리즘을 사용하는 것을 넘어, 데이터 자체의 품질과 관리 방식에 크게 좌우됩니다.

- **데이터 품질의 중요성:**
 - **한 줄 핵심 요약:** 빅 데이터 시대에는 데이터의 양보다 질이 훨씬 중요합니다.
 - **직관적 예시/비유:** 아무리 많은 재료가 있어도 상한 재료가 많으면 맛있는 요리를 만들 수 없는 것과 같습니다. 모델은 '쓰레기를 넣으면 쓰레기가 나온다(Garbage In, Garbage Out)'는 원칙을 따릅니다.
 - **기술적 정확한 설명:** 모호하지 않고, 잘 정제되고, 신뢰할 수 있는 레이블을 가진 '고품질 데이터(High Quality Data)'는 모델의 성능을 크게 향상시킵니다. 데이터 포인트가 아무리 많아도 모두 노이즈(noise)라면 모델은 신호(signal)를 구분하지 못하고 혼란에 빠집니다.
- **데이터 중심 접근 vs. 코드/알고리즘 중심 접근:**
 - **한 줄 핵심 요약:** 모델 성능 개선을 위해 데이터 자체를 개선하는 '데이터 중심(Data-centric)' 접근 방식이 알고리즘만 개선하는 '코드 중심(Code-centric)' 접근보다 효과적일 수 있습니다.
 - **직관적 예시/비유:** 망치질이 잘 안 될 때, 망치 잡는 법(알고리즘)만 계속 바꾸기보다 못이 휘었는지(데이터) 확인하고 똑바른 못을 쓰는 것이 더 나은 해결책일 수 있습니다.
 - **기술적 정확한 설명:** AutoML과 같은 도구는 알고리즘을 최적화하지만, 데이터의 품질을 직접적으로 제어하지는 못합니다. 따라서 데이터의 불균형, 노이즈, 편향 등을 직접 개선하는 것이 모델 성능 향상에 더 큰 영향을 미칩니다.
- **비즈니스 요구사항의 중요성:**
 - **한 줄 핵심 요약:** 모든 데이터 과학 또는 ML 프로젝트는 명확한 비즈니스 문제 해결을 목표로 해야 합니다.
 - **직관적 예시/비유:** 멋진 도구를 만들었지만 아무도 필요로 하지 않는다면 의미가 없습니다. ML 모델도 마찬가지로 실제 문제를 해결해야 가치가 있습니다.
 - **기술적 정확한 설명:** 데이터와 ML 기술에 대한 매료만으로 프로젝트를 시작하는 것이 아니라, 비즈니스 요구사항을 명확히 이해하고 모델이 어떻게 수렴해야 하는지, 어떤 평가 지표를 사용해야 하는지 등을 결정해야 합니다. 연구 프로젝트의 경우에도 명확한 목표와 해결하려는 문제가 있어야 합니다.
- **모델 추론 유형 (Model Inferencing):**
 - **한 줄 핵심 요약:** 학습된 모델은 배치, 스트리밍, 실시간 API의 세 가지 주요 방식으로 실제 환경에서 사용될 수 있습니다.
 - **기술적 정확한 설명:**
 - * **배치 추론 (Batch Inferencing):** 대량의 데이터를 한 번에 모아 주기적으로(예: 하루에 한 번) 모델을 실행하여 결과를 생성합니다.
 - * **스트리밍 추론 (Streaming Inferencing):** 데이터가 들어오는 즉시 실시간으로 모델을 실행하여 점수를 매기고, 결과를 저장하거나 하위 시스템으로 전달합니다.
 - * **실시간 API 기반 추론 (Real-time API-based Inferencing):** 웹 애플리케이션과 같이 낮은 지연 시간(low latency)과 높은 탄력성(elasticity)이 요구되는 환경에서 REST API 엔드포인트를

통해 모델을 호출하여 즉각적인 예측을 얻습니다.

- **앙상블 모델 (Ensemble Models):**

- 한 줄 핵심 요약: 여러 모델의 예측을 결합하여 단일 모델보다 더 강건하고 정확한 예측을 얻는 기법입니다.
- 기술적 정확한 설명: 각 모델의 단점을 다른 모델이 보완함으로써 전반적인 모델의 강건성(robustness)과 정확도(accuracy)를 향상시킵니다. AutoML이 여러 모델을 동시에 실행하는 것도 앙상블의 한 형태입니다.

- **모델 오류 (Model Errors):**

- 한 줄 핵심 요약: 모든 모델은 부동 소수점(floating point) 계산과 데이터의 본질적인 무작위성 때문에 항상 100% 완벽할 수 없으며, 약간의 오차를 허용해야 합니다.
- 기술적 정확한 설명: 이로 인해 모델의 재현성(reproducibility)이 어려워지며, 예측 오차는 편향(bias), 분산(variance), 비환원 오차(irreducible error)의 세 가지 유형으로 구성됩니다.

- **하이퍼파라미터 튜닝 (Hyperparameter Tuning):**

- 한 줄 핵심 요약: 모델의 성능을 최적화하기 위해 하이퍼파라미터의 최적 조합을 찾는 과정입니다.
- 기술적 정확한 설명: 과거에는 '그리드 서치(Grid Search)'와 같은 방식이 사용되었으나, 최근에는 '베이저안 검색(Bayesian Search)'과 같은 확률적 검색 방법이 Optuna, Hyperopt 등의 라이브러리를 통해 널리 사용됩니다. 베이저안 검색은 머신러닝을 사용하여 다음 탐색할 파라미터 공간을 예측함으로써 훨씬 빠르게 최적점에 수렴합니다.

- **ML 파이프라인 (ML Pipeline):**

- 한 줄 핵심 요약: 데이터 전처리부터 모델 학습까지의 일련의 단계를 체계적으로 구성한 것입니다.
- 기술적 정확한 설명: ML 파이프라인은 크게 두 가지 유형의 스테이지로 구성됩니다.
 - * **변환기 (Transformer):** 데이터프레임을 입력받아 다른 데이터프레임을 출력합니다. (예: 특징 스케일링, 결측치 처리)
 - * **추정기 (Estimator):** 데이터프레임을 입력받아 학습된 모델을 출력합니다. (예: 분류기, 회귀 모델)

변환기는 transform 메서드를, 추정기는 fit 또는 predict 메서드를 호출합니다.

- **데이터 버전 관리 (Data Versioning):**

- 한 줄 핵심 요약: 데이터의 변경 이력을 자동으로 기록하고 특정 시점의 데이터를 복원할 수 있도록 하는 기능입니다.
- 기술적 정확한 설명: Delta Lake와 같은 기술은 데이터에 대한 모든 작업(업데이트, 삭제 등)마다 명시적인 버전 번호와 타임스탬프를 자동으로 부여합니다. 이를 통해 개발자는 특정 버전의 데이터를 쉽게 조회하거나 복원할 수 있으며, 이를 '타임 트래블(Time Travel)' 기능이라고도 합니다.

AutoML의 이해: 블랙박스 vs. 글래스박스

AutoML은 기계 학습 모델 개발 과정을 자동화하여, 전문 지식이 없는 사용자도 쉽게 모델을 생성할 수 있도록 돕는 기술입니다.

- **AutoML의 정의 및 목적:**

- **한 줄 핵심 요약:** 사용자가 데이터만 제공하면, 다양한 통계적 특성, 알고리즘, 하이퍼파라미터 튜닝 등을 자동으로 처리하여 최적의 ML 모델을 찾아주는 시스템입니다.
- **직관적 예시/비유:** 복잡한 요리 레시피를 몰라도 재료만 넣으면 자동으로 맛있는 요리를 만들어주는 스마트 오븐과 같습니다.
- **기술적 정확한 설명:** 사용자는 데이터만 지정하고 클릭하면 모델을 얻을 수 있습니다. 이는 ML 전문가가 아닌 '시민 데이터 과학자(Citizen Data Scientist)'도 생산적으로 작업할 수 있게 합니다.

- **블랙박스 AutoML의 한계:**

- **한 줄 핵심 요약:** 초기 AutoML 솔루션(예: DataRobot)은 모델 생성 과정을 숨겨, 내부 작동 방식을 알 수 없는 '블랙박스(Blackbox)' 형태로 제공되었습니다.
- **직관적 예시/비유:** 스마트 오븐이 요리를 잘 만들지만, 어떤 재료를 어떻게 섞고 어떤 온도로 구웠는지 전혀 알 수 없는 상태입니다.
- **기술적 정확한 설명:** 대규모 기업 환경에서는 모델의 내부를 들여다보고 변경해야 할 필요성이 생기는데, 블랙박스 모델은 이러한 요구사항을 충족시키지 못했습니다. 모델이 왜 특정 예측을 했는지, 어떤 부분을 개선해야 하는지 알 수 없어 결국 한계에 부딪혔습니다.

- **글래스박스 AutoML의 특징 (Databricks AutoML):**

- **한 줄 핵심 요약:** Databricks AutoML은 모델 생성 과정의 모든 단계를 사용자에게 투명하게 공개하는 '글래스박스(Glassbox)' 접근 방식을 채택합니다.
- **직관적 예시/비유:** 스마트 오븐이 요리를 만드는 모든 과정을 레시피와 함께 상세히 기록하여 보여주는 것과 같습니다. 사용자는 이 레시피를 보고 수정하거나 개선할 수 있습니다.
- **기술적 정확한 설명:**
 - * **가시성 및 투명성:** 사용자를 대신하여 수행된 모든 작업(노트북, 하이퍼파라미터 튜닝, 탐색적 데이터 분석(EDA), 모델 반복, 최적 모델 선정 및 정당화)이 가시적으로 제공됩니다.
 - * **MLflow 통합:** MLflow와 깊이 통합되어 모델 추적(tracking), 레지스트리(registry) 등 모든 MLflow 기능이 함께 제공됩니다.
 - * **노트북 제공:** 최적 모델을 포함한 모든 실험 과정의 노트북 코드가 제공되어 사용자가 이를 검토, 수정, 재사용할 수 있습니다. **활용성:**
 - **데이터 유효성 검사:** 데이터가 예측 시나리오에 적합한 신호를 가지고 있는지 빠르게 판단할 수 있습니다.
 - **EDA 및 프로파일링:** 데이터 프로파일링 및 EDA 단계를 자동화하여 초기 분석 시간을 단축합니다.
 - **모델 해석 가능성:** SHAP과 같은 도구를 사용하여 모델의 예측을 설명하고 특징 중요도를 파악할 수 있습니다.
 - **생산 배포:** 생성된 모델을 모델 레지스트리에 등록하고 추론 모드에서 쉽게 로드하여 사용할 수 있습니다.
 - * **제한된 모델 수:** DataRobot과 같은 다른 제공업체에 비해 제공되는 모델의 종류는 적지만, 핵심적인 모델에 집중하여 투명성을 높입니다.

- * **시민 데이터 과학자 친화적:** ML에 대한 깊은 지식이 없는 사용자도 쉽게 모델을 개발하고 활용할 수 있도록 설계되었습니다.
- * **대규모 데이터셋 처리:** 분산 엔진을 사용하여 매우 큰 데이터셋에 대해서도 AutoML을 실행할 수 있습니다.
- **Databricks AutoML 지원 모델 유형:**
 - **분류 모델 (Classification Models):**
 - * **목적:** 데이터를 미리 정의된 범주(클래스) 중 하나로 분류합니다. (예: 암/비암, 비/비비) 이진 분류 또는 다중 클래스 분류가 가능합니다.
 - * **지원 알고리즘:** 결정 트리 (Decision Trees), 랜덤 포레스트 (Random Forest), 로지스틱 회귀 (Logistic Regression), XGBoost, LightGBM.
 - * **작동 방식:** 다양한 하이퍼파라미터 조합으로 이 모델들을 병렬로 실행하여 가장 효율적인 결과를 도출합니다. 사용자는 최대 대기 시간(예: 1시간)을 설정할 수 있습니다.
 - **회귀 모델 (Regression Models):**
 - * **목적:** 연속적인 값을 예측합니다. (예: 내일 금값, 한 달 후 부동산 가격)
 - * **지원 알고리즘:** 결정 트리, 랜덤 포레스트, 선형 회귀 (Linear Regression), XGBoost, LightGBM.
 - **예측 모델 (Forecasting Models):**
 - * **목적:** 시계열 데이터를 기반으로 미래 값을 예측합니다.
 - * **지원 알고리즘:** Prophet, Auto ARIMA, DPR.

주의사항

Databricks 무료 에디션의 제약 사항: 강의에서 사용된 서버리스 무료 에디션은 컴퓨팅 자원과 사용 가능한 모델이 제한적입니다. 특히 예측 모델만 지원하며, 데이터셋이 매우 작고 데이터 내 신호가 명확해야만 성공적으로 실행될 수 있습니다. 대규모 데이터셋에서는 시간 초과가 발생할 수 있습니다.

모델 오류의 종류: 편향, 분산, 비환원 오차

모델의 예측 오차는 세 가지 주요 구성 요소로 나눌 수 있으며, 이들을 이해하는 것은 모델 성능 개선에 필수적입니다.

– 편향 (Bias):

- * **한 줄 핵심 요약:** 모델의 예측값과 실제(기대)값 사이의 체계적인 차이입니다. 주로 모델이 너무 단순하여 데이터의 복잡한 패턴을 학습하지 못하는 '과소적합(Underfitting)' 상태에서 발생합니다.
- * **직관적 예시/비유:** 시험 공부를 너무 적게 해서 시험 문제의 핵심을 파악하지 못하고 낮은 점수를 받는 것과 같습니다.
- * **기술적 정확한 설명:** ML 모델이 특징과 목표(target) 간의 진정한 관계를 포착하지 못할 때 발생합니다. 훈련 시간을 늘리거나 모델의 복잡도를 높이면 편향을 줄일 수 있습니다.

– 분산 (Variance):

- * **한 줄 핵심 요약:** 모델이 훈련 데이터의 작은 변화에도 너무 민감하게 반응하는 정도입니다. 주로 모델이 너무 복잡하여 훈련 데이터에 과도하게 맞춰지는 '과대적합(Overfitting)' 상태에서 발생합니다.
- * **직관적 예시/비유:** 시험 공부를 너무 많이 해서 모든 예시 문제와 답을 외웠지만, 실제 시험에서 문제가 조금만 다르게 나오면 전혀 풀지 못하는 것과 같습니다.
- * **기술적 정확한 설명:** 모델이 훈련 데이터의 노이즈까지 학습하여 새로운 데이터에 대한 일반화 능력이 떨어집니다. 모델의 복잡도를 줄이거나 더 많은 데이터를 사용하여 분산을 줄일 수 있습니다.

– 비환원 오차 (Irreducible Error):

- * **한 줄 핵심 요약:** 데이터 자체의 본질적인 무작위성이나 측정 오차 등으로 인해 모델이 아무리 완벽해도 줄일 수 없는 최소한의 오차입니다.
- * **직관적 예시/비유:** 아무리 정확한 온도계라도 미세한 환경 변화 때문에 항상 100% 정확한 온도를 측정할 수 없는 것과 같습니다.
- * **기술적 정확한 설명:** 모델의 제어 범위를 벗어나는 요인(예: 부동 소수점 계산의 한계, 데이터 수집 과정의 무작위성)으로 인해 발생하며, 모델의 예측 오차는 편향, 분산, 비환원 오차의 합으로 구성됩니다.

– 편향-분산 트레이드오프 (Bias-Variance Trade-off):

- * **한 줄 핵심 요약:** 편향을 줄이면 분산이 증가하고, 분산을 줄이면 편향이 증가하는 경향이 있습니다. 이 둘 사이의 균형을 찾는 것이 중요합니다.
- * **직관적 예시/비유:** 한쪽으로 너무 치우치면 다른 쪽이 부족해지는 시소와 같습니다.
- * **기술적 정확한 설명:** 모델의 복잡도를 높이면 편향은 줄어들지만(과소적합 해소), 분산은 증가할 수 있습니다(과대적합 발생). 반대로 모델을 단순화하면 분산은 줄어들지만 편향이 증가할 수 있습니다.

Figure 1: 편향과 분산의 시각적 예시 (타겟 비유)

- * **높은 편향, 낮은 분산:** 모든 예측이 한 곳에 뭉쳐 있지만, 실제 타겟에서 멀리 떨어져 있습니다. (과소적합)
- * **낮은 편향, 높은 분산:** 예측들이 타겟 주변에 넓게 퍼져 있습니다. (과대적합)
- * **높은 편향, 높은 분산:** 예측들이 타겟에서 멀리 떨어져 넓게 퍼져 있습니다. (최악의 경우)

- * 낮은 편향, 낮은 분산: 예측들이 타겟의 중앙에 가깝게 뭉쳐 있습니다. (이상적인 경우)

모델 편향의 원인과 감소 방법

모델 편향은 단순히 기술적인 문제뿐만 아니라 인간의 인지 편향이 데이터에 반영되어 발생하기도 합니다.

* 모델 편향의 원인:

- 인간의 인지 편향 (Human Cognitive Bias): 모델은 인간이 제공한 데이터를 학습하므로, 데이터에 내재된 인간의 편향을 그대로 모방합니다.
- 데이터의 불충분한 대표성 (Underrepresented Data): 특정 그룹 (예: 어두운 피부색의 여성)에 대한 훈련 데이터가 부족하면, 모델은 해당 그룹에 대해 낮은 인식 정확도를 보입니다.
- 낮은 품질의 훈련 데이터 (Poor Quality Training Data):
- 데이터 자체의 품질 문제: 흐릿하거나 왜곡된 이미지처럼 데이터 자체의 품질이 낮으면 모델이 정확하게 학습하기 어렵습니다.
- 잘못된 레이블링 (Mislabeling): 데이터에 잘못된 레이블이 부여되면 모델은 잘못된 정보를 학습하여 편향된 예측을 합니다. (예: '악마'라는 단어에 소수 인종의 이미지가 연결되는 경우)
- 모델 성능 불일치 (Model Performance Mismatch): 모델이 학습한 데이터와 실제 적용될 데이터의 특성이 다를 때 발생합니다.
- 데이터 조작 (Data Manipulation): 의도적으로 편향된 데이터를 주입하여 모델의 학습 과정을 왜곡시킬 수 있습니다. (예: 홀로코스트가 없었다는 정보를 주입하여 모델이 환각을 일으키는 경우)

* 모델 편향 감소 방법:

- 모델 변경 및 적절한 모델 사용:
- 트리 기반 알고리즘(Tree algorithms)은 편향 처리에 더 효과적일 수 있습니다.
- 데이터의 특징과 목표 간에 선형 관계가 없다면 선형 모델(Linear model)을 사용하지 않는 등 문제에 적합한 모델을 선택해야 합니다.
- 데이터의 대표성 확보:
- 훈련 데이터가 모든 가능한 그룹과 결과를 다양하게 대표하도록 보장해야 합니다.
- 데이터가 불균형하다면 가중치(weighting)를 부여하거나 페널티(penalized) 모델을 사용합니다. (예: 사기 거래와 같이 매우 적은 비율의 데이터에 더 큰 가중치를 부여)
- 광범위한 하이퍼파라미터 튜닝: 최적의 하이퍼파라미터 조합을 찾아 모델이 데이터의 패턴을 더 잘 학습하도록 합니다.
- 앙상블 모델 사용: 여러 모델을 결합하여 단일 모델의 편향을 상쇄하고 전반적인 예측 성능을 향상시킵니다.

분산 감소 방법

- * 앙상블 (Ensemble): 여러 모델을 훈련시켜 예측을 결합함으로써 모델의 분산을 줄입니다. 약한 학습자(weak learners)와 강한 학습자(strong learners)를 모두 활용하여 예측 성능을 개선합니다.
- * 더 큰 데이터셋으로 훈련: 더 많은 데이터를 사용하면 데이터 대 노이즈 비율(data to noise ratio)이 증가하여 모델의 분산이 감소하고, 모델이 새로운 데이터에 더 잘 일반화될 수 있습니다.

일반적인 모델 오류 처리 전략

* 올바른 학습 모델 선택:

- 지도 학습 (Supervised Learning): 데이터셋을 준비한 이해관계자가 레이블의 정확성을 완전히 제어해야 합니다. 잘못된 레이블은 모델 오류로 직결됩니다.
- 비지도 학습 (Unsupervised Learning): 신경망 자체의 품질과 구축 방식에 따라 모델의 성능이 달라집니다.

* 데이터셋 재사용 금지: 민족적으로 다양한 인구 집단의 데이터는 단일 인종이 주로 거주하는 지역에 적용될 수 없습니다. 데이터셋의 특성이 적용될 환경과 일치해야 합니다.

* 신중한 데이터 처리: 데이터 전처리 과정(정제, 변환, 검증, 정규화)을 신중하게 수행하고, 필요한 경우 추가 검증이나 후처리 해석을 적용해야 합니다.

* 실세계 성능 모니터링: 모델을 실제 데이터에 대해 테스트하고, 성능을 지속적으로 모니터링하여 문제가 발생하면 즉시 대응합니다.

* 훈련 세트 빈번한 변경: 데이터 특성이 시간에 따라 변할 수 있으므로, 훈련 세트를 자주 업데이트하여 모델이 최신 경향을 반영하도록 합니다.

* 인프라 문제 해결: AutoML 훈련 시 컴퓨팅 자원 부족과 같은 인프라 문제가 모델 오류로 이어질 수 있으므로, 이를 적시에 파악하고 해결해야 합니다.

* 데이터 수집 프로세스 검토: 데이터 정제, 변환, 검증, 정규화 등 데이터 수집 과정 전반에 걸쳐 철저한 조사가 필요합니다.

데이터 불균형 문제와 해결 전략

데이터 불균형 (Imbalanced Data)은 기계 학습에서 흔히 발생하는 문제로, 특정 클래스의 데이터 수가 다른 클래스에 비해 현저히 적을 때 발생합니다.

* 데이터 불균형의 정의 및 발생 원인:

- **한 줄 핵심 요약:** 훈련 데이터에서 하나 이상의 클래스가 다른 클래스에 비해 매우 적은 비율을 차지하는 현상입니다.
- **직관적 예시/비유:** 100개의 사과 중 99개가 빨간 사과이고 1개만 파란 사과라면, 모델은 파란 사과를 잘 인식하지 못할 것입니다.
- **기술적 정확한 설명:** 대부분의 기계 학습 알고리즘은 클래스 분포가 균등하다고 가정하고 정확도를 최대화하도록 설계되어 있습니다. 사기 탐지 (fraud detection), 암 진단 (cancer detection), 이상 탐지 (anomaly detection)와 같은 분야에서 흔히 발생합니다.

* 데이터 불균형의 영향:

- **한 줄 핵심 요약:** 모델이 다수 클래스에 편향되어 소수 클래스를 제대로 예측하지 못하게 됩니다.
- **기술적 정확한 설명:** 모델은 다수 클래스를 정확하게 예측하는 데 집중하여 전체 정확도는 높게 나오지만, 실제 중요한 소수 클래스 (예: 사기, 암)에 대한 예측 성능 (재현율 등)은 매우 낮아집니다.

* 데이터 불균형 탐지 방법:

- **한 줄 핵심 요약:** 종속 변수 (target variable)의 범주형 값 (categorical values) 별 개수를 확인합니다.
- **기술적 정확한 설명:** `value_counts()` , `(majorityclass)` `(minorityclass)` .

* 데이터 불균형 해결 전략:

- **성능 지표 보강 (Augment Performance Metrics):**
 - **한 줄 핵심 요약:** 정확도 (Accuracy) 만으로는 불균형 데이터셋의 모델 성능을 제대로 평가할 수 없으므로, 다른 지표들을 함께 사용해야 합니다.
 - **기술적 정확한 설명:** 혼동 행렬 (Confusion Matrix), 정밀도 (Precision), 재현율 (Recall), F1 점수 (F1 Score) 등을 사용하여 모델의 완전성 (completeness)과 민감도 (sensitivity)를 평가합니다.
- **다른 알고리즘 사용 (Different Algorithms):**
 - **한 줄 핵심 요약:** 트리 기반 모델 (예: 결정 트리, 랜덤 포레스트)은 불균형 데이터에 더 강건한 경향이 있습니다.
 - **기술적 정확한 설명:** 특정 알고리즘은 불균형 데이터셋에서 더 나은 성능을 보일 수 있으므로, 다양한 알고리즘을 시도해 보는 것이 좋습니다.
- **재샘플링 기법 (Resampling Techniques):**
 - **한 줄 핵심 요약:** 소수 클래스의 데이터 수를 늘리거나 (오버샘플링), 다수 클래스의 데이터 수를 줄여 (언더샘플링) 클래스 분포를 균등하게 만듭니다.
 - **기술적 정확한 설명:**
 - **오버샘플링 (Oversampling):** 소수 클래스의 샘플을 복제하거나 합성하여 데이터 수를 늘립니다. 과도하게 복제하면 과대적합의 위험이 있습니다.
 - **언더샘플링 (Undersampling):** 다수 클래스의 샘플을 무작위로 제거하여 데이터 수를 줄입니다.

- 다. 중요한 정보가 손실될 위험이 있습니다.
- **주의사항:** 재샘플링 기법을 적용하기 전에 데이터를 훈련 세트와 테스트 세트로 분할해야 합니다. 테스트 세트에는 원본 데이터의 불균형을 유지하여 모델의 실제 성능을 평가해야 합니다.
 - **SMOTE (Synthetic Minority Over-sampling Technique):**
 - **한 줄 핵심 요약:** 소수 클래스의 기존 샘플을 기반으로 새로운 '합성(synthetic)' 샘플을 생성하여 오버샘플링하는 고급 기법입니다.
 - **기술적 정확한 설명:**
 1. **최근접 이웃 찾기:** 소수 클래스 샘플(검은색 원)에 대해 N개의 최근접 이웃을 찾습니다.
 2. **선 연결:** 해당 소수 클래스 샘플과 그 이웃들 사이에 선을 그립니다.
 3. **합성 샘플 생성:** 이 선 위에서 무작위 지점을 선택하여 새로운 합성 데이터 포인트를 생성합니다.

이 방법은 단순히 기존 샘플을 복제하는 것보다 더 다양한 데이터를 생성하여 모델의 일반화 성능을 향상시킵니다. `imblearn` 라이브러리에서 `SMOTE` 함수를 사용할 수 있습니다.

Figure 2: SMOTE를 이용한 합성 샘플 생성 원리

- **합성 데이터 생성의 어려움:** SMOTE와 같은 기법으로 합성 샘플을 생성하는 것은 단순히 무작위 값을 생성하는 것보다 훨씬 복잡하며, 데이터의 통계적 특성을 유지하면서 의미 있는 샘플을 만들어야 합니다. Databricks Lab DataGen과 같은 도구는 이러한 복잡성을 해결하기 위해 개발되었습니다.

민감 데이터 분류 및 익명성 보장

모델이 민감한 개인 식별 정보(PII)를 유출하는 것을 방지하고 데이터의 익명성을 보장하는 것은 매우 중요합니다.

· PII 유출 위험:

- **한 줄 핵심 요약:** 모델이 훈련 데이터에 포함된 주민등록번호, 전화번호와 같은 민감 정보를 학습하여, 질문에 따라 해당 정보를 유출할 수 있습니다.
- **직관적 예시/비유:** 친구에게 비밀을 말했는데, 그 친구가 다른 사람에게 그 비밀을 그대로 말해버리는 것과 같습니다.
- **기술적 정확한 설명:** 모델에 민감 정보가 포함된 데이터를 그대로 학습시키면, 모델은 해당 정보를 기억하고 추론 과정에서 이를 노출할 수 있습니다. 이는 심각한 개인정보 보호 문제를 야기합니다.

· 데이터 분류의 필요성:

- **한 줄 핵심 요약:** 모델에 데이터를 공급하기 전에 민감 정보를 자동으로 탐지하고 분류하여 제거하거나 보호해야 합니다.
- **기술적 정확한 설명:**
- **자동화된 메커니즘:** 정규 표현식(regular expressions)과 같은 수동적인 방법으로는 대규모 데이터셋에서 민감 정보를 정확하게 탐지하기 어렵습니다. 따라서 지능적인 데이터 분류 메커니즘이 필요합니다.

· Databricks의 데이터 분류 기능:

- **활성화:** Databricks 워크스페이스 및 카탈로그 수준에서 데이터 분류 기능을 활성화할 수 있습니다.
- **자동 스캔:** 활성화되면 카탈로그 내의 모든 스키마와 테이블을 자동으로 스캔하여 민감 정보를 탐지합니다.
- **지역별 특화:** 전화번호, 주민등록번호 등은 국가별로 형식이 다르므로, 미국과 같이 특정 지역에 특화된 15 20가지 일반적인 민감 정보를 탐지합니다. 향후 사용자 정의 규칙도 지원될 예정입니다.
- **결과 확인:** 스캔이 완료되면 위치, 이름, 전화번호, 이메일 주소, 은행 계좌 번호, IP 주소, 운전면허증 등 탐지된 민감 정보 유형과 해당 컬럼 수를 대시보드에서 확인할 수 있습니다.

실제 적용 사례: 대규모 사기 탐지 시스템

Eric이 과거에 참여했던 신용카드 거래 사기 탐지 시스템 구축 프로젝트는 데이터 엔지니어링과 데이터 과학이 결합된 실제 대규모 시스템의 좋은 예시입니다.

비즈니스 요구사항

이 시스템은 기존 시스템의 단점을 보완하고 다음과 같은 핵심 요구사항을 충족해야 했습니다.

- **높은 신뢰도의 사기 탐지:**
- **목표:** 거래가 사기인지 아닌지를 높은 신뢰도로 판단해야 합니다.
- **중요성:** 실제 사기를 놓치면 수천 달러 이상의 손실이 발생하고, 합법적인 거래를 사기로 오인하면 고객 경험이 심각하게 손상될 수 있습니다.
- **낮은 지연 시간 (Low Latency):**
- **목표:** 거래 요청 후 15밀리초(ms) 이내에 사기 여부를 응답해야 합니다.
- **중요성:** 신용카드 거래는 잔액 확인 등 여러 단계가 동시에 진행되므로, 사기 탐지에는 극히 짧은 시간이 할당됩니다. 일반적인 디스크 읽기 시간보다도 짧은 매우 엄격한 요구사항입니다.
- **높은 처리량 (High Throughput):**
- **목표:** 초당 50,000 건 이상의 거래를 처리할 수 있어야 합니다.
- **중요성:** 전 세계적으로 발생하는 방대한 거래량을 감당하기 위한 필수적인 요구사항입니다.
- **설정 및 관리 용이한 규칙 및 특징 관리:**
- **목표:** 데이터 과학자들이 새로운 특징(feature)과 규칙(rule)을 쉽게 정의하고 관리하며 배포할 수 있어야 합니다.
- **중요성:** 사기 패턴은 계속 진화하므로, 시스템이 새로운 위협에 신속하게 대응할 수 있도록 유연성이 필요합니다. 기존 시스템은 특징 변경이 매우 어려웠습니다.
- **새로운 특징의 신속한 배포:**
- **목표:** 새로운 특징이 추가될 경우, 과거 데이터에 대한 특징 값을 빠르게 계산하고 실시간 추론에 활용할 수 있어야 합니다.
- **중요성:** 새로운 특징의 효과를 검증하고 실제 시스템에 적용하는 데 걸리는 시간을 최소화해야 합니다.

특징(Feature) 공학

특징 공학은 모델의 정확도에 결정적인 영향을 미치는 매우 중요한 과정입니다.

- **특징의 중요성:**
- **한 줄 핵심 요약:** 거래 및 차원(dimension) 속성을 기반으로 모델 학습에 필요한 유의미한 특징을 계산합니다.
- **기술적 정확한 설명:** 특징은 원시 데이터에서 직접 얻는 속성(attribute)과 달리, 다른 속성이나 특징을 기반으로 계산된 값입니다.
- **주요 차원 (Dimensions):**
- 고객(Customers), 계정(Accounts), 판매자(Merchants), 위치(Locations), 시간(Time of day), 제품(Product), 가격(Price) 등 거래와 관련된 다양한 엔티티들이 차원으로 활용됩니다.
- **거래 특징 (Transaction Features) 예시:**

- **집으로부터의 거리:** 거래가 발생한 판매자의 위치와 고객의 거주지 간의 거리. (예: 고객이 캠브리지에 사는데 이스탄불에서 거래 발생)
- **평균/최대/최소/평균과의 차이:** 고객의 평소 거래 금액과 현재 거래 금액의 차이. (예: 평소 20달러 거래하던 고객이 갑자기 20,000달러 거래)
- **일일 거래 횟수:** 고객의 하루 평균 거래 횟수와 현재 거래 횟수의 차이. (예: 하루 12건 거래하던 고객이 갑자기 100건 거래)
- **차원 특징 (Dimension Features) 예시:**
 - 판매자별 평균 거래 금액: 특정 판매자에서 발생하는 평균 거래 금액.
 - 소비자/판매자별 거래 빈도: 특정 소비자 또는 판매자에서 발생하는 거래의 빈도.
- 수백 가지 특징 지원: 데이터 과학 팀은 수백 가지의 다양한 특징을 개발했으며, 시스템은 이 모든 특징을 계산하고 관리할 수 있어야 했습니다.

시스템 아키텍처

이 시스템은 람다 아키텍처(Lambda Architecture)를 기반으로 실시간 및 배치 처리를 결합하여 복잡한 요구사항을 충족했습니다.

Figure 3: 사기 탐지 시스템 아키텍처 개요

- **메타데이터 모델:**
 - **한 줄 핵심 요약:** 시스템의 모든 엔티티(이벤트, 차원, 속성, 특징, 규칙, ML 모델)에 대한 정보를 정의하고 관리하는 모델입니다.
- **기술적 정확한 설명:**
 - **이벤트 (Events):** 거래(transactions)와 같은 사실 데이터. 계좌 개설/폐쇄와 같은 다른 유형의 이벤트도 포함될 수 있습니다.
 - **차원 (Dimensions):** 고객, 판매자, 위치 등 이벤트와 관련된 컨텍스트 정보. 새로운 차원(예: 달의 위상)도 쉽게 추가할 수 있도록 확장 가능하게 설계되었습니다.
 - **속성 (Attributes):** 거래 금액, 시간과 같이 거래나 차원에 직접 포함된 정적 값입니다.
 - **특징 (Features):** 다른 속성이나 특징을 기반으로 계산된 값입니다. 특징은 다른 특징 위에 구축될 수 있습니다.
 - **규칙 (Rules):** 사기 탐지를 위한 'if-then-else' 형태의 복잡한 수동 정의 규칙입니다.
 - **ML 모델 (ML Models):** 고전적인 머신러닝 모델과 신경망(Neural Network) 기술을 포함합니다.
 - **멀티테넌트 (Multi-tenant):** 여러 결제 처리업체, 판매자, 고객을 지원하도록 설계되어, 데이터 모델에 가시성과 거버넌스(governance)를 내장했습니다.
- **메타데이터 서비스 (Metadata Service):**
 - **한 줄 핵심 요약:** 위에서 정의된 모든 메타데이터 정보를 관리하는 서비스입니다.
 - **기술적 정확한 설명:** 특징, 규칙, ML 모델과 같이 자주 변경되는 메타데이터를 쉽게 업데이트할 수 있도록 지원합니다.
 - **코드 생성 (Code Generation):**
 - **한 줄 핵심 요약:** 메타데이터를 기반으로 특징 계산 코드를 자동으로 생성하는 혁신적인 기술

입니다.

- **직관적 예시/비유:** 특징의 정의(예: '지난 한 달간 고객의 평균 거래 금액')만 입력하면, 이 특징을 계산하는 SQL 코드나 EPL(Complex Event Processing Language) 코드를 자동으로 만들어주는 공장과 같습니다.
- **기술적 정확한 설명:**
 - **자동화된 특징 계산:** 각 특징에 대한 코드를 수동으로 작성하는 대신, 메타데이터를 사용하여 스트리밍 환경과 배치 환경 모두에서 특징을 계산하는 코드를 생성합니다.
 - **유지보수 용이성:** 새로운 특징을 추가하거나 기존 특징을 변경할 때, 코드 생성기를 통해 쉽게 업데이트할 수 있어 인적 오류를 줄이고 개발 속도를 높입니다.
 - **회귀 방지:** 코드 생성기가 변경될 경우, 기존 특징들이 올바르게 작동하는지 확인하는 회귀 테스트(regression testing)가 중요합니다.
- **람다 아키텍처 (Lambda Architecture):**
 - **한 줄 핵심 요약:** 실시간 스트리밍 처리와 배치 처리를 동시에 사용하여 데이터 처리의 효율성과 정확성을 극대화하는 아키텍처입니다.
 - **기술적 정확한 설명:**
 - **실시간 처리 (Streaming Service):**
 - **구성:** 특징 계산, ML 모델 실행, 규칙 처리 등 모든 과정이 15ms 이내에 완료됩니다.
 - **역할:** 들어오는 거래에 대해 즉시 사기 여부를 판단하여 응답합니다.
 - **코드 생성 활용:** 특징 계산 코드는 스트리밍 코드 생성기를 통해 생성됩니다.
 - **배치 처리 (Batch Processing):**
 - **구성:** Hadoop 클러스터를 사용하여 대량의 과거 거래 데이터를 저장하고 처리합니다.
 - **역할:** 새로운 특징이 추가될 경우, 과거 데이터에 대한 특징 값을 계산하고 Cassandra 데이터베이스에 저장하여 실시간 시스템에서 참조할 수 있도록 합니다. 모델 훈련 및 테스트에도 사용됩니다.
 - **코드 생성 활용:** 특징 계산 코드는 배치 코드 생성기를 통해 생성됩니다.
 - **데이터 저장 및 관리 (Cassandra):**
 - **한 줄 핵심 요약:** 실시간 특징 값 검색을 위해 높은 성능과 확장성을 제공하는 NoSQL 데이터 베이스인 Cassandra를 사용했습니다.
 - **기술적 정확한 설명:**
 - **성능:** 2.5ms의 빠른 읽기/쓰기 속도를 제공하여 15ms의 지연 시간 요구사항을 충족하는 데 기여했습니다. Cassandra는 쓰기보다 읽기가 느린 특성이 있지만, 충분히 빨랐습니다.
 - **확장성:** 페타바이트(PB) 규모로 수평 확장이 가능하여 대규모 데이터를 처리할 수 있습니다. (단, 비용 고려 필요)
 - **데이터 모델:**
 - **이벤트 테이블 (Event Table):** 모든 거래(이벤트)의 속성과 계산된 특징 값을 저장하는 팩트 테이블(fact table) 역할을 합니다.
 - **차원 테이블 (Dimension Table):** 고객, 판매자, 터미널(카드 리더기), 계정 등 다양한 차원 정보를 저장합니다. Cassandra의 컬럼형(columnar) 특성 덕분에 희소 데이터(sparse data)를 효율적으로 관리할 수 있었습니다.

- **유연한 스키마:** 새로운 이벤트 유형, 특징 유형, 차원 유형, 속성 등을 쉽게 추가할 수 있는 유연한 스키마를 가졌습니다.
- **순환 버퍼 (Circular Buffer / Ring Buffer):**
- **한 줄 핵심 요약:** WPI 학생들과 개발한 특허 기술로, 고정된 시간 간격(예: 1주, 1일) 동안의 데이터에 대한 카운트(count)와 합계(sum)만 저장하여 특징 값을 효율적으로 계산하고 저장하는 방법입니다.
- **직관적 예시/비유:** 매일의 거래 내역을 모두 저장하는 대신, '오늘 총 몇 건의 거래가 있었고 총 금액은 얼마인지'만 기록하고, 일주일의 지나면 가장 오래된 날의 기록을 지우고 최신 날의 기록을 추가하는 방식입니다.
- **기술적 정확한 설명:**
- **효율적인 저장:** 수백만 고객의 개별 거래 내역을 모두 저장하는 대신, 각 시간 버킷(예: 일요일, 월요일 등)별로 카운트와 합계만 유지합니다.
- **다양한 특징 계산:** 이 카운트와 합계만으로 평균, 최대, 최소, 표준편차 등 다양한 특징 값을 매우 효율적으로 계산할 수 있습니다.
- **유연한 시간 간격:** 7일, 30일, 24시간 등 다양한 시간 간격으로 설정할 수 있습니다.
- **빠른 검색:** 개별 거래를 조회할 필요 없이 버퍼에서 직접 특징 값을 빠르게 검색할 수 있습니다.
- **비용 절감:** 대규모 저장 공간이 필요 없으므로 온프레미스(on-premise) 환경에서 비용 절감을 크게 기여했습니다.

시스템 견고성 및 재해 복구

실제 운영 환경에서는 시스템의 안정성과 재해 복구 능력이 매우 중요합니다.

- **데이터 센터 이중화 (Data Center Redundancy):**
- **한 줄 핵심 요약:** 시스템 장애에 대비하여 미국과 유럽에 각각 데이터 센터를 구축하고, 한쪽이 실패하면 자동으로 다른 쪽으로 트래픽을 전환하는 기능을 구현했습니다.
- **기술적 정확한 설명:**
- **자동 전환:** 한 데이터 센터에 장애가 발생하면 모든 트래픽이 다른 데이터 센터로 자동 전환되어 서비스 중단을 방지합니다.
- **지연 시간 증가 가능성:** 전환 시 일시적으로 지연 시간 요구사항을 충족하지 못할 수 있지만, 서비스 연속성을 보장하는 것이 더 중요했습니다.
- **복잡성:** 두 시스템 간의 특징 값 동기화 및 데이터 프라이버시(GDPR) 준수 등 복잡한 문제가 발생할 수 있습니다.
- **종합적인 프레임워크:**
- 복잡한 이벤트 처리 프레임워크를 통해 실시간으로 특징을 계산하고 모델 및 규칙을 적용하여 결과를 반환합니다.
- Cassandra 데이터베이스는 빠른 데이터 검색을 제공하며, 독점적인 순환 버퍼 기술은 특징 저장 및 계산의 효율성을 높였습니다.
- Hadoop 클러스터는 배치 처리를 통해 새로운 특징을 추가하고, 과거 데이터를 기반으로 모델을 구축 및 테스트하며, 시스템 장애 시 데이터를 복구하는 데 사용됩니다.

실습/코드/오류와 해결

Databricks AutoML 사용법 (분류 모델 예시)

Databricks AutoML을 사용하여 분류 모델을 생성하는 과정은 매우 간단하며, 대부분의 작업이 자동으로 수행됩니다.

1. **AutoML 실험 시작:**
2. Databricks 워크스페이스에서 '실험(Experiments)' 섹션으로 이동합니다.
3. '분류 AutoML(Classification AutoML)'을 선택합니다.
4. **클러스터 선택:** ML 런타임 클러스터를 지정합니다.
5. **훈련 데이터셋 지정:** 3단계 이름 공간(카탈로그.스키마.테이블)을 사용하여 훈련 데이터를 선택합니다. (예: churn 데이터셋)
6. **예측 컬럼 지정:** 모델이 예측할 목표 컬럼을 선택합니다. (예: churn이 'yes'인지 'no'인지)
7. **실험 이름 부여:** 실험에 고유한 이름을 지정합니다. (예: churn-am)
8. **컬럼 포함/제외:** 기본적으로 모든 컬럼이 포함되지만, 식별자나 불필요한 날짜 필드 등은 제외할 수 있습니다.
9. **결측치 처리 (Impute with):** 결측치 처리 방식을 선택합니다. (예: 최적 추측 사용, 상수 값, 최빈값)
10. **고급 설정 (Advanced Configuration):**
11. **주요 평가 지표 (Primary Evaluation Metric):** 모델 성능 평가를 위한 주요 지표를 선택합니다. (예: F1 점수)
12. **훈련 프레임워크 (Training Frameworks):** 사용할 ML 프레임워크를 지정합니다.
13. **최대 훈련 시간 (Maximum Training Time):** AutoML이 모델을 훈련할 최대 시간을 설정합니다. (예: 15분)
14. **실험 실행 및 결과 확인:**
15. 'AutoML 시작(Start AutoML)' 버튼을 클릭하면 실험이 시작됩니다.
16. **EDA 노트북:** AutoML은 데이터 탐색 노트북을 자동으로 생성하여 데이터 프로파일링, 결측치 확인, 변수 통계 등을 시각적으로 보여줍니다.
17. **최적 모델 노트북:** F1 점수와 같은 지정된 평가 지표에 따라 가장 좋은 성능을 보인 모델(예: LightGBM)에 대한 전체 코드가 담긴 노트북을 제공합니다. 이 노트북에는 다음 내용이 포함됩니다.
18. 데이터 로드 및 전처리 (불리언, 수치형, 범주형 컬럼 분리)
19. 훈련, 검증, 테스트 세트 분할
20. LightGBM 모델 훈련 및 하이퍼파라미터 튜닝 (Hyperopt 사용)
21. MLflow를 통한 실험 추적
22. SHAP을 이용한 특징 중요도 (Feature Importance) 및 모델 해석 가능성
23. 모델 레지스트리 등록 및 추론 모드에서 모델 로드 예시 코드
24. 혼동 행렬, 정밀도, 재현율 등 다양한 평가 지표 시각화
25. **MLflow 통합:** 모든 실험 실행은 MLflow에 자동으로 기록되어, 각 실행의 메트릭, 파라미터, 아티팩트(모델 파일, 환경 정보 등)를 확인할 수 있습니다.

□ 예제

Databricks AutoML 실행 예시 (가상 코드)

```

1 from databricks import automl
2
3 # 훈련데이터셋지정카탈로그스키마테이블    (...)
4 training_data = "your_catalog.your_schema.churn_data"
5
6 # 예측할타겟컬럼
7 target_col = "churn"
8
9 # 실험이름
10 experiment_name = "churn-prediction-automl"
11
12 # AutoML 실행
13 summary = automl.classify(
14     training_data=training_data,
15     target_col=target_col,
16     experiment_name=experiment_name,
17     timeout_minutes=15, # 최대분 15 훈련
18     primary_metric="f1_score", % F1 점수를주요평가지표로사용
19     % exclude_cols=["customer_id", "date_field"] % 제외할컬럼
20 )
21
22 # 최적모델노트북보기
23 print(f"Best model notebook: {summary.best_model.notebook_path}")
24
25 # 데이터탐색노트북보기
26 print(f>Data exploration notebook: {summary.
    data_exploration_notebook_path}")

```

Listing 1: Databricks AutoML 실행 코드 스니펫

데이터 불균형 처리 코드 예시

신용카드 사기 탐지 데이터셋을 사용하여 데이터 불균형 문제를 확인하고, 재샘플링 기법으로 이를 해결하는 과정을 살펴봅니다.

- (a) 라이브러리 설치 및 데이터 로드:
- (b) imblearn (imbalanced-learn) 라이브러리를 설치합니다.
- (c) Kaggle에서 얻은 신용카드 거래 데이터를 로드합니다.

```

1 %pip install imblearn scikit-learn pandas numpy
2
3 import pandas as pd
4 from sklearn.model_selection import train_test_split
5 from sklearn.linear_model import LogisticRegression
6 from sklearn.metrics import accuracy_score, f1_score,
    recall_score, confusion_matrix

```

```

7 from imblearn.over_sampling import RandomOverSampler
8 from imblearn.under_sampling import RandomUnderSampler
9 from imblearn.over_sampling import SMOTE
10
11 # 가상의데이터로드실제 ( 데이터는 Kaggle 등에서얻어야함 )
12 # df = pd.read_csv("creditcard.csv")
13 # 예시를위한가상데이터생성
14 data = {
15     'V1': [i * 0.1 for i in range(100000)],
16     'V2': [i * 0.05 for i in range(100000)],
17     'Amount': [i % 1000 for i in range(100000)],
18     'Class': [0] * 99900 + [1] * 100 % 는99.9% 0, 는0.1% 1 사기()
19 }
20 df = pd.DataFrame(data)
21
22 X = df.drop('Class', axis=1)
23 y = df['Class']
24
25 X_train, X_test, y_train, y_test = train_test_split(X, y,
26     test_size=0.3, random_state=42, stratify=y)
27
28 print("원본 훈련데이터클래스분포 :")
29 print(y_train.value_counts())
30 % 예상출력 :
31 % 0      69930
32 % 1       70
33 % Name: Class, dtype: int64

```

Listing 2: Imblearn 라이브러리 설치 및 데이터 로드

- (d) 불균형 데이터에서의 모델 성능 확인:
- (e) 더미 분류기 (Dummy Classifier): 실제 알고리즘이 아닌, 단순히 다수 클래스를 예측하는 더미 분류기를 사용해도 정확도는 매우 높게 나올 수 있습니다. 이는 정확도만으로는 불균형 데이터셋의 성능을 제대로 평가할 수 없음을 보여줍니다.
- (f) 로지스틱 회귀 (Logistic Regression): 불균형 데이터셋에 로지스틱 회귀를 적용하면, 정확도는 높지만 소수 클래스(사기)에 대한 재현율(Recall)이 매우 낮게 나올 수 있습니다.

```

1 # 로지스틱회귀모델훈련및예측
2 model = LogisticRegression(solver='liblinear', random_state=42)
3 model.fit(X_train, y_train)
4 y_pred = model.predict(X_test)
5
6 print("\n불균형n 데이터로지스틱회귀성능 :")
7 print(f"정확도 (Accuracy): {accuracy_score(y_test, y_pred):.4f}")
8 print(f"F1 점수 (F1 Score): {f1_score(y_test, y_pred):.4f}")
9 print(f"재현율 (Recall): {recall_score(y_test, y_pred):.4f}")
10 print("혼동 행렬:\n", confusion_matrix(y_test, y_pred))
11 % 예상출력재현율이 ( 낮을수있음 ):
12 % 정확도 (Accuracy): 0.9990

```

```

13 % F1 점수 (F1 Score): 0.6400
14 % 재현율 (Recall): 0.6400
15 % 혼동행렬 :
16 % [[29970    0]
17 % [   18    32]]

```

Listing 3: 불균형 데이터에서의 로지스틱 회귀 성능

(g) 오버샘플링 (Oversampling):

(h) 한 줄 핵심 요약: 소수 클래스의 샘플 수를 다수 클래스와 비슷하게 늘려 데이터 불균형을 해소합니다.

(i) 기술적 정확한 설명: RandomOverSampler를 사용하여 소수 클래스(사기)의 샘플을 복제하여 훈련 세트의 클래스 분포를 균등하게 만듭니다.

```

1 # 를 RandomOverSampler 이용한 오버샘플링
2 ros = RandomOverSampler(random_state=42)
3 X_resampled_ros, y_resampled_ros = ros.fit_resample(X_train,
4 y_train)
5 print("\오버샘플링 후 훈련 데이터 클래스 분포 :")
6 print(y_resampled_ros.value_counts())
7 % 예상 출력 :
8 % 0    69930
9 % 1    69930
10 % Name: Class, dtype: int64
11
12 # 오버샘플링된 데이터로 모델 훈련 및 예측
13 model_ros = LogisticRegression(solver='liblinear', random_state
14 =42)
15 model_ros.fit(X_resampled_ros, y_resampled_ros)
16 y_pred_ros = model_ros.predict(X_test)
17 print("\오버샘플링 후 로지스틱 회귀 성능 :")
18 print(f"정확도 (Accuracy): {accuracy_score(y_test, y_pred_ros):.4f}")
19 print(f"F1 점수 (F1 Score): {f1_score(y_test, y_pred_ros):.4f}")
20 print(f"재현율 (Recall): {recall_score(y_test, y_pred_ros):.4f}")
21 print("혼동 행렬:\n", confusion_matrix(y_test, y_pred_ros))
22 % 재현율이 개선될 수 있음

```

Listing 4: RandomOverSampler를 이용한 오버샘플링

(j) 언더샘플링 (Undersampling):

(k) 한 줄 핵심 요약: 다수 클래스의 샘플 수를 소수 클래스와 비슷하게 줄여 데이터 불균형을 해소합니다.

(l) 기술적 정확한 설명: RandomUnderSampler를 사용하여 다수 클래스(비사기)의 샘플을 무작위로 제거하여 훈련 세트의 클래스 분포를 균등하게 만듭니다.

```

1 # 를 RandomUnderSampler 이용한 언더샘플링
2 rus = RandomUnderSampler(random_state=42)

```

```

3 X_resampled_rus, y_resampled_rus = rus.fit_resample(X_train,
4   y_train)
5 print("\n언더샘플링n 후훈련데이터클래스분포      :")
6 print(y_resampled_rus.value_counts())
7 % 예상출력 :
8 % 0      70
9 % 1      70
10 % Name: Class, dtype: int64
11
12 # 언더샘플링된데이터로모델훈련및예측
13 model_rus = LogisticRegression(solver='liblinear', random_state
14   =42)
15 model_rus.fit(X_resampled_rus, y_resampled_rus)
16 y_pred_rus = model_rus.predict(X_test)
17
18 print("\n언더샘플링n 후로지스틱회귀성능      :")
19 print(f"정확도 (Accuracy): {accuracy_score(y_test, y_pred_rus):.4f}")
20 print(f"F1 점수 (F1 Score): {f1_score(y_test, y_pred_rus):.4f}")
21 print(f"재현율 (Recall): {recall_score(y_test, y_pred_rus):.4f}")
22 print("혼동 행렬:\n", confusion_matrix(y_test, y_pred_rus))
23 % 재현율이개선될수있지만      , 정확도는낮아질수있음

```

Listing 5: RandomUnderSampler를 이용한 언더샘플링

(m) SMOTE (Synthetic Minority Over-sampling Technique):

- (n) 한 줄 핵심 요약: 소수 클래스의 기존 샘플을 기반으로 새로운 합성 샘플을 생성하여 오버샘플링합니다.
- (o) 기술적 정확한 설명: SMOTE를 사용하여 소수 클래스의 샘플을 복제하는 대신, 통계적 특성을 고려한 합성 샘플을 생성하여 훈련 세트의 클래스 분포를 균등하게 만듭니다.

```

1 # 를SMOTE 이용한합성데이터생성
2 smote = SMOTE(random_state=42)
3 X_resampled_smote, y_resampled_smote = smote.fit_resample(
4   X_train, y_train)
5
6 print("\nSMOTE 후훈련데이터클래스분포      :")
7 print(y_resampled_smote.value_counts())
8 % 예상출력 :
9 % 0      69930
10 % 1      69930
11 % Name: Class, dtype: int64
12
13 # 된SMOTE 데이터로모델훈련및예측
14 model_smote = LogisticRegression(solver='liblinear',
15   random_state=42)
16 model_smote.fit(X_resampled_smote, y_resampled_smote)
17 y_pred_smote = model_smote.predict(X_test)
18
19 print("\nSMOTE 후로지스틱회귀성능      :")

```

```

18 print(f"정확
    도 (Accuracy): {accuracy_score(y_test, y_pred_smote):.4f}")
19 print(f"F1 점
    수 (F1 Score): {f1_score(y_test, y_pred_smote):.4f}")
20 print(f"재현
    율 (Recall): {recall_score(y_test, y_pred_smote):.4f}")
21 print("혼동 행렬:\n", confusion_matrix(y_test, y_pred_smote))
22 % 재현율과 F1 점수가 더욱 개선될 수 있음

```

Listing 6: SMOTE를 이용한 합성 데이터 생성

주의사항

합성 데이터 생성의 어려움: SMOTE와 같은 합성 데이터 생성 기법은 단순히 무작위 값을 생성하는 것이 아닙니다. 데이터의 통계적 분포와 특징 간의 관계를 고려하여 실제와 유사한 샘플을 만들어야 합니다. Databricks Lab DataGen과 같은 전문 도구는 이러한 복잡성을 해결하기 위해 개발되었으며, JP Morgan Chase와 같은 기업에서 테스트 환경의 데이터 생성을 위해 사용됩니다.

체크리스트

다음 체크리스트를 통해 강의 내용을 얼마나 이해했는지 스스로 점검해 보세요.

26. 데이터 품질: 빅 데이터보다 고품질 데이터가 중요한 이유를 설명할 수 있는가?
27. 데이터 중심 ML: 데이터 중심 접근 방식이 코드 중심 접근 방식보다 유리한 경우를 설명할 수 있는가?
28. 비즈니스 요구사항: ML 프로젝트에서 비즈니스 요구사항이 왜 가장 먼저 고려되어야 하는지 설명할 수 있는가?
29. 모델 추론 유형: 배치, 스트리밍, 실시간 API 추론의 차이점을 설명할 수 있는가? **AutoML**: 블랙박스 AutoML과 글래스박스 AutoML의 차이점을 설명할 수 있는가?
30. **Databricks AutoML**: Databricks AutoML이 제공하는 투명성 기능(노트북, MLflow 통합)을 설명할 수 있는가?
31. 모델 오류: 편향, 분산, 비환원 오차의 정의와 발생 원인(과소적합/과대적합)을 설명할 수 있는가?
32. 편향-분산 트레이드오프: 편향과 분산이 서로 상충 관계에 있음을 설명할 수 있는가?
33. 모델 편향 원인: 인간의 인지 편향, 데이터 불균형, 낮은 데이터 품질이 모델 편향으로 이어지는 과정을 설명할 수 있는가?
34. 데이터 불균형: 데이터 불균형이 모델 성능에 미치는 영향과 이를 탐지하는 방법을 설명할 수 있는가?
35. 불균형 데이터 해결: 오버샘플링, 언더샘플링, SMOTE의 원리와 적용 방법을 설명할 수 있는가?
36. 민감 데이터: PII 유출 위험과 데이터 분류의 중요성을 설명할 수 있는가?
37. 사기 탐지 시스템: Eric의 사례 연구에서 대규모 사기 탐지 시스템의 주요 비즈니스 요구사항과 기술적 해결책(람다 아키텍처, 코드 생성, 순환 버퍼, Cassandra)을 설명할 수 있는가?

FAQ

초심자들이 자주 혼동하거나 궁금해할 만한 질문과 답변을 정리했습니다.

38. **Q: AutoML이 모든 것을 자동으로 해주는데, 왜 데이터 과학자가 여전히 필요한가요?**
39. **A:** AutoML은 모델 개발의 반복적이고 표준적인 부분을 자동화하여 효율성을 높이지만, 데이터의 품질을 개선하거나, 복잡한 비즈니스 문제를 정의하고, 모델의 예측을 해석하며, 윤리적 편향을 해결하는 등의 고차원적인 작업은 여전히 데이터 과학자의 전문 지식을 필요로 합니다. 글래스박스 AutoML은 이러한 전문가의 개입을 용이하게 합니다.
- Q: 모델의 정확도가 99%인데도 문제가 될 수 있나요?**
40. **A:** 네, 특히 데이터 불균형이 심한 경우 문제가 될 수 있습니다. 예를 들어, 1000건의 거래 중 1건만 사기인 경우, 모든 거래를 '사기 아님'으로 예측해도 정확도는 99.9%가 됩니다. 하지만 실제 사기를 전혀 탐지하지 못하므로, 이는 매우 심각한 문제입니다. 이럴 때는 정확도 외에 재현율, 정밀도, F1 점수와 같은 다른 지표를 함께 고려해야 합니다.
41. **Q: 편향과 분산 중 어느 것을 먼저 해결해야 하나요?**
42. **A:** 일반적으로 모델이 너무 단순하여 데이터의 패턴을 전혀 학습하지 못하는 '높은 편향(과소적합)' 상태라면 편향을 먼저 줄이는 데 집중합니다. 모델이 어느 정도 학습을 시작한 후, 훈련 데이터에 너무 맞춰져 새로운 데이터에 약한 '높은 분산(과대적합)' 상태라면 분산을 줄이는 데 집중합니다. 이 둘 사이의 균형을 찾는 것이 중요합니다.
43. **Q: 합성 데이터를 생성하는 것이 왜 그렇게 어려운가요?**
44. **A:** 단순히 무작위 값을 생성하는 것은 쉽지만, 실제 데이터의 통계적 특성, 특징 간의 상관관계, 그리고 클래스 분포를 정확하게 반영하는 '의미 있는' 합성 데이터를 생성하는 것은 매우 복잡합니다. 잘못된 합성 데이터는 모델을 오도하여 성능을 저하시킬 수 있습니다. SMOTE와 같은 기법은 이러한 통계적 고려사항을 바탕으로 합성 샘플을 생성합니다.
45. **Q: Lambda 아키텍처 대신 Kappa 아키텍처를 사용하면 안 되나요?**
46. **A:** Kappa 아키텍처는 모든 데이터를 스트리밍 방식으로 처리하여 배치 레이어를 제거함으로써 시스템을 단순화합니다. 하지만 당시 Eric의 프로젝트에서는 실시간 처리 서비스의 기술적 한계로 마이크로배치(microbatch) 처리가 충분히 정교하지 못했습니다. 따라서 배치 처리의 장점(과거 데이터 재계산, 모델 훈련)과 스트리밍 처리의 장점(실시간 응답)을 모두 활용하는 Lambda 아키텍처가 더 적합했습니다. 현대에는 Spark Streaming과 같은 기술로 Kappa 아키텍처 구현이 더 용이해졌습니다.

빠르게 훑어보기 (1 페이지 요약)

CSCI103 Lecture 10 핵심 요약

1. 데이터 품질의 중요성:

47. 빅 데이터 → 고품질 데이터: 양보다 질이 중요. 데이터 중심 접근이 모델 성능 개선에 효과적.

48. 비즈니스 요구사항: 모든 ML 프로젝트는 명확한 비즈니스 문제 해결을 목표로 해야 함.

2. AutoML의 이해:

49. 블랙박스 ML: 내부 작동 방식 불투명 (DataRobot). 대규모 기업에서 한계.

50. 글래스박스 ML: 모든 과정 투명하게 공개 (Databricks AutoML). 노트북, MLflow 통합, 모델 해석 가능. 활용: 데이터 유효성 검사, EDA 자동화, 시민 데이터 과학자 지원.

3. 모델 오류의 종류:

51. 편향 (Bias): 과소적합 (Underfitting) → 모델이 데이터 패턴을 충분히 학습 못함.

52. 분산 (Variance): 과대적합 (Overfitting) → 모델이 훈련 데이터에 너무 맞춰짐.

53. 비환원 오차: 데이터 자체의 무작위성으로 인한 최소 오차.

54. 편향-분산 트레이드오프: 둘은 상충 관계. 균형이 중요.

55. 편향 원인: 인간의 인지 편향, 데이터 불균형, 낮은 데이터 품질, 잘못된 레이블링.

4. 데이터 불균형 문제:

56. 정의: 특정 클래스 데이터 수가 현저히 적음 (사기, 암 진단).

57. 영향: 모델이 다수 클래스에 편향되어 소수 클래스 예측 성능 저하.

58. 해결 전략:

59. 성능 지표 보강 (정확도 외 F1, 재현율, 정밀도).

60. 재샘플링 (오버샘플링, 언더샘플링).

61. SMOTE: 소수 클래스 합성 샘플 생성.

5. 민감 데이터 분류:

62. PII 유출 위험: 모델이 민감 정보(주민번호, 전화번호)를 학습하여 노출 가능.

63. 자동 분류: Databricks의 카탈로그 수준 데이터 분류 기능으로 민감 정보 자동 탐지 및 보호.

6. 대규모 사기 탐지 시스템 (Eric 사례):

64. 요구사항: 높은 신뢰도, 15ms 지연 시간, 50,000 TPS 처리량, 유연한 특징/규칙 관리.

65. 아키텍처:

66. 람다 아키텍처: 실시간 스트리밍 + 배치 처리.

67. 코드 생성: 메타데이터 기반 특징 계산 코드 자동 생성 (스트리밍/배치).

68. 데이터 저장: Cassandra (빠른 읽기/쓰기, 수평 확장성).

69. 순환 버퍼: WPI 특허 기술. 특징 값 효율적 저장/계산.

70. 견고성: 데이터 센터 이중화, 재해복구.